

Comprendre le code — TP1/TP6

Authentification MFA, moteur RBAC/ABAC et audit — explication fonction par fonction

*Ce que fait chaque fonction, et surtout **pourquoi** elle a été écrite ainsi*

Ce document parcourt ton projet fichier par fichier. Pour chaque fonction : ce qu'elle fait, et surtout la raison de son existence — pourquoi elle est séparée des autres, quel problème elle résout, quelle règle de sécurité elle traduit. L'objectif n'est pas de mémoriser le code, mais de comprendre la logique pour pouvoir l'expliquer et l'étendre toi-même (notamment pour TP7).

Comment lire ce document

L'ordre suit le flux réel d'une requête dans l'application : d'abord la couche **db/** (comment on parle à MongoDB), puis **auth/** (qui es-tu ?), puis **access/** (as-tu le droit ?), puis **audit/** (on garde la preuve), et enfin **main.py** qui orchestre tout ça dans l'application Flask.

db/ — Accès aux données MongoDB [3 fichiers]

Ce module isole tout ce qui touche à MongoDB. Aucun autre fichier du projet n'ouvre de connexion MongoDB directement — tout passe par ici. Si un jour tu changes de base de données, c'est le seul endroit à modifier.

`db/connection.py`

```
get_db()
```

Rôle

Retourne l'instance de la base MongoDB, en la créant une seule fois (singleton via la variable globale `_client`).

Pourquoi cette fonction existe séparément

Créer une connexion MongoClient a un coût (ouverture de socket, négociation). Si chaque fonction du projet créait sa propre connexion, l'application ouvrirait des dizaines de connexions inutiles. Le singleton garantit qu'une seule connexion est réutilisée partout.

```
get_collection(name)
```

Rôle

Raccourci qui renvoie directement une collection (ex. "users", "resources", "alerts") sans que l'appelant ait à écrire `get_db()[name]` à chaque fois.

Pourquoi cette fonction existe séparément

Toutes les autres fonctions du projet (`audit/logger.py`, `access/engine.py`, `main.py`...) appellent `get_collection("...")` plutôt que de manipuler `get_db()` directement — un seul point d'entrée, plus facile à tester et à faire évoluer.

`db/models.py`

```
make_user(user_id, name, role, department, password_hash, totp_secret,
mfa_enabled, clearance)
```

Rôle

Construit le dictionnaire exact d'un document utilisateur à insérer dans MongoDB.

Pourquoi cette fonction existe séparément

MongoDB n'impose aucun schéma par défaut : rien n'empêche d'insérer des documents avec des champs différents d'un utilisateur à l'autre. Cette fonction joue le rôle d'un schéma implicite — tant que tout le monde insère via `make_user()`, la forme des documents reste garantie identique.

```
make_resource(resource_id, patient_id, rtype, owner_department,
sensitivity)
```

Rôle

Même principe que `make_user`, pour les documents ressource.

Pourquoi cette fonction existe séparément

Ces champs sont exactement ceux que `access/engine.py` lit pour prendre sa décision (`type`, `owner_department`, `sensitivity`) — centraliser leur construction évite qu'un import CSV mal formé casse silencieusement le moteur de décision.

```
make_log(user_id, role, department, resource_id, resource_type,  
sensitivity, action, ip, success, mfa_passed, reason)
```

Rôle

Construit une entrée standard du journal d'audit, avec un timestamp généré automatiquement.

Pourquoi cette fonction existe séparément

C'est le contrat utilisé par `audit/logger.py` à chaque écriture. En centralisant sa construction ici plutôt que dans `logger.py` directement, on sépare « quelle est la forme d'un log » (`models.py`) de « quand écrit-on un log » (`logger.py`) — les deux peuvent évoluer indépendamment.

À retenir

Remarque : le timestamp est généré ici, pas transmis en paramètre — impossible pour un appelant de falsifier l'heure d'un log, ce qui protège l'intégrité de la preuve.

`db/mongo_client.py`

Fichier actuellement vide dans ton projet : il est réservé à TP7, pour y ajouter une fonction de création d'index MongoDB sur la future collection **alerts** (voir le prompt d'implémentation TP7 fourni séparément).

auth/ — Authentification (qui es-tu ?) [3 fichiers]

Ce module répond uniquement à la question de l'identité : mot de passe correct ? code TOTP valide ? Il ne contient **aucune** logique de permission — c'est intentionnel : mélanger « qui es-tu » et « as-tu le droit » rendrait le code plus difficile à auditer et à faire évoluer séparément.

auth/security.py

hash_password(plain)

Rôle

Transforme un mot de passe en clair en un hash bcrypt (avec sel aléatoire intégré).

Pourquoi cette fonction existe séparément

Le sujet l'exige explicitement : un mot de passe ne doit jamais être stocké en clair. bcrypt intègre un salage automatique, ce qui protège même si deux utilisateurs choisissent le même mot de passe.

verify_password(plain, hashed)

Rôle

Vérifie qu'un mot de passe saisi correspond au hash stocké, sans jamais reconstituer le mot de passe original.

Pourquoi cette fonction existe séparément

bcrypt est un hachage à sens unique : on ne peut pas « dé-hacher ». La seule façon de vérifier est de demander à bcrypt de comparer directement — d'où une fonction dédiée plutôt qu'une comparaison de chaînes classique.

generate_totp_secret()

Rôle

Génère un secret aléatoire (base32) qui servira à calculer les codes TOTP d'un utilisateur.

Pourquoi cette fonction existe séparément

Si ce secret était partagé entre utilisateurs, un code volé à l'un compromettrait tous les autres. Une fonction dédiée garantit que chaque appel produit un secret réellement unique et aléatoire.

get_totp_uri(secret, username, issuer)

Rôle

Construit l'URI otpauth:// standard, utilisable pour générer un QR code d'enrôlement dans une application d'authentification.

Pourquoi cette fonction existe séparément

C'est un format standard (Google Authenticator, Authy...) : autant réutiliser le format plutôt que d'inventer un mécanisme propriétaire.

verify_totp(secret, code)

Rôle

Vérifie qu'un code à 6 chiffres saisi correspond bien au secret de l'utilisateur, à l'instant présent.

Pourquoi cette fonction existe séparément

C'est la fonction qui matérialise le "second facteur" : même si le mot de passe est volé, l'attaquant doit aussi connaître ce secret pour produire le bon code.

À retenir

Le paramètre valid_window=0 signifie qu'aucune tolérance n'est acceptée sur la fenêtre de temps — un choix strict qui peut provoquer des refus si l'horloge du serveur et du client divergent.

`get_current_totp(secret)`

Rôle

Renvoie le code TOTP valide à l'instant présent pour un secret donné.

Pourquoi cette fonction existe séparément

Dans ce projet, il n'y a pas de vraie application TOTP côté utilisateur : ce code est directement envoyé par email (voir `auth/mailler.py`) ou affiché dans le terminal en secours — cette fonction sert à produire ce code à afficher/envoyer.

`create_jwt(user_id, role, department, mfa_ok)`

Rôle

Émet un jeton signé (JWT) qui prouve l'identité de l'utilisateur pour les requêtes suivantes.

Pourquoi cette fonction existe séparément

Sans JWT, il faudrait revérifier mot de passe + OTP à chaque requête. Le JWT sépare clairement authentification (une fois, au login) et autorisation (à chaque requête, via `access/engine.py`) — le JWT ne dit jamais « autorisé » ou « refusé », seulement « voici qui tu es et si ton MFA est validé ».

`decode_jwt(token)`

Rôle

Décode et valide la signature et l'expiration d'un JWT.

Pourquoi cette fonction existe séparément

Sans cette vérification, n'importe qui pourrait fabriquer un faux jeton prétendant être admin. La levée d'exception (`jwt.ExpiredSignatureError`, `jwt.InvalidTokenError`) en cas de problème force l'appelant à gérer explicitement le cas "non authentifié".

`auth/mailler.py`

`send_otp_email(user_id, code, minutes_validite)`

Rôle

Envoie le code OTP par email via le serveur SMTP de Gmail, à une adresse fixe (OTP_RECIPIENT).

Pourquoi cette fonction existe séparément

Simule ici le "canal du second facteur" (au lieu d'une vraie app TOTP sur smartphone). Retourne `False` si la configuration SMTP est absente, ce qui permet à `main.py` de retomber proprement sur un affichage du code dans le terminal plutôt que de planter.

`auth/routes.py` — API JSON pure

`_get_user(user_id)`

Rôle

Recherche un utilisateur par identifiant dans MongoDB.

Pourquoi cette fonction existe séparément

Petite fonction privée pour éviter de répéter `find_one(...)` dans chaque route de ce fichier.

`login()` – POST /auth/login

Rôle

Vérifie le mot de passe ; si l'utilisateur a la MFA activée, répond "mfa_required" sans encore émettre de JWT ; sinon émet directement un JWT avec mfa_ok=False.

Pourquoi cette fonction existe séparément

Cette route API est indépendante du site web (main.py a sa propre version avec sessions et flash messages) : elle sert un client externe (script, application mobile) qui dialogue en JSON pur.

`verify_otp()` – POST /auth/verify-otp

Rôle

Vérifie le code OTP, puis émet un JWT complet avec mfa_ok=True.

Pourquoi cette fonction existe séparément

Deuxième étape obligatoire de l'authentification forte côté API — sans cet appel, impossible d'obtenir un JWT avec mfa_ok=True, donc impossible d'accéder à une ressource sensible.

`totp_debug(user_id)` – GET /auth/totp-debug/<user_id>

Rôle

Affiche dans le terminal (et par email) le code TOTP courant d'un utilisateur donné.

Pourquoi cette fonction existe séparément

Route utilitaire de développement/test. **Point de vigilance** : cette route ne vérifie aucune identité avant de révéler le code — utile en développement, mais à retirer ou protéger avant toute mise en production.

access/ — Autorisation (as-tu le droit ?) [3 fichiers]

C'est le cœur sécurité du projet : une fois l'identité confirmée par auth/, ce module décide si l'action demandée est réellement permise, en combinant RBAC (rôle), ABAC (contexte) et MFA.

[access/policy.py](#) — la politique, sous forme de données

```
POLICY = { rôle: { action: [types de ressources autorisés] } }
```

Rôle

Dictionnaire déclaratif qui traduit directement la matrice RBAC du sujet (médecin, infirmier, secrétaire, admin_securite, patient × read/write/create/export).

Pourquoi cette fonction existe séparément

Écrire la politique comme une **donnée** plutôt que comme une série de "if" dispersés dans le code présente deux avantages : elle est relisible d'un coup d'œil pour un audit de sécurité, et modifiable sans toucher à la logique du moteur de décision.

```
MFA_REQUIRED_SENSITIVITIES, DEPARTMENT_RESTRICTED_ROLES, HOUR_MIN, HOUR_MAX
```

Rôle

Constantes nommées : quels niveaux de sensibilité exigent la MFA, quels rôles sont soumis à la contrainte de département, quelle est la plage horaire de service.

Pourquoi cette fonction existe séparément

Nommer ces valeurs évite les "nombres magiques" (pourquoi 6 ? pourquoi 20 ?) épars dans le code, et centralise les réglages que le sujet pourrait faire évoluer (ex. élargir la plage horaire).

[access/engine.py](#) — le moteur de décision

```
authorize(user, resource, action, context, ip) -> (bool, str)
```

Rôle

Applique, dans l'ordre, 5 vérifications fail-fast : (1) MFA sur ressource sensible, (2) l'action existe-t-elle pour ce rôle, (3) le type de ressource est-il autorisé pour cette action, (4) le département correspond-il (ABAC), (5) l'heure est-elle dans la plage de service. Journalise systématiquement le résultat, puis renvoie (autorisé/refusé, justification).

Pourquoi cette fonction existe séparément

L'ordre n'est pas arbitraire : on vérifie d'abord la fiabilité de l'identité (MFA) avant même d'évaluer des permissions plus fines — inutile d'étudier le rôle de quelqu'un dont on n'est pas sûr de l'identité. Regrouper les 5 vérifications dans une seule fonction garantit aussi qu'aucune route de l'application ne peut "oublier" une étape : tout accès passe par ce point unique.

À retenir

Le type de retour tuple[bool, str] force chaque appelant à toujours recevoir une justification, jamais seulement un booléen — cette justification est ce qui rend le journal exploitable.

`_write_log(user, resource, action, ip, success, mfa_passed, reason)` (privée)

Rôle

Appelle systématiquement `audit.logger.log_access()` avec les bons champs, sur chaque branche de `authorize()` — succès comme échec.

Pourquoi cette fonction existe séparément

En la faisant appeler sur **chaque** retour de `authorize()` (pas seulement à la fin), on garantit qu'un refus ne peut jamais "sortir" de la fonction sans laisser de trace. C'est la traduction directe de la règle du sujet : « il faut journaliser aussi les accès refusés ».

`access/routes.py` — exposition API des ressources

`_current_user(token)` (privée)

Rôle

Décode le JWT, va chercher le document utilisateur complet en base, et y injecte le statut `mfa_ok` extrait du jeton.

Pourquoi cette fonction existe séparément

Chaque route protégée de ce fichier a besoin exactement de cette même opération — la centraliser évite trois copier-coller quasi identiques et donc trois endroits à corriger en cas de bug.

`_bearer(request_obj)` (privée)

Rôle

Extrait le jeton du header HTTP Authorization: Bearer <token>.

Pourquoi cette fonction existe séparément

Standard HTTP pour transmettre un jeton ; factoriser son extraction évite de répéter le même découpage de chaîne dans chaque route.

`get_resource(resource_id)` – GET `/access/resource/<id>`

Rôle

Décode l'identité, récupère la ressource, appelle `authorize()` avec l'action "read", et ne renvoie la ressource que si l'accès est autorisé.

Pourquoi cette fonction existe séparément

Aucune donnée n'est jamais renvoyée sans passer explicitement par `authorize()` — impossible de "contourner" le moteur de décision depuis cette route.

`write_resource(resource_id)` – PUT `/access/resource/<id>`

Rôle

Même principe que `get_resource`, mais pour l'action "write" : vérifie l'autorisation avant d'appliquer la mise à jour demandée.

Pourquoi cette fonction existe séparément

Symétrie volontaire avec `get_resource` : chaque type d'action (lecture, écriture) repasse par la même vérification, avec seulement le paramètre "action" qui change.

export_journal() – GET /access/export/journal

Rôle

Construit une "fausse" ressource représentant le journal d'accès lui-même, puis appelle `authorize()` avec l'action "export" avant de renvoyer les logs.

Pourquoi cette fonction existe séparément

Astuce de conception notable : plutôt que d'écrire une vérification de rôle ad hoc pour cette route spéciale, elle réutilise exactement le même moteur de décision que toutes les autres routes — même l'équipe sécurité (`admin_securite`) est soumise à une vérification auditable et cohérente.

audit/ — La preuve [2 fichiers]

Ce module ne contient aucune logique de décision : il se contente d'écrire, de façon fiable, ce qui s'est passé. C'est le pilier « Accounting » du modèle AAA et « Preuve » du modèle DICP.

```
log_access(user_id, role, department, resource_id, resource_type,
sensitivity, action, ip, success, mfa_passed, reason) – audit/logger.py
```

Rôle

Construit une entrée via `db.models.make_log()` et l'insère dans la collection `access_logs`.

Pourquoi cette fonction existe séparément

Fonction unique et systématiquement appelée par `access/engine.py`, qu'il s'agisse d'un succès ou d'un refus. Le `try/except` autour de l'insertion garantit qu'une panne MongoDB n'empêche jamais l'application de répondre à l'utilisateur — l'erreur est seulement signalée en console ([AUDIT ERROR]), jamais remontée en exception bloquante.

audit/alert_writer.py est actuellement un fichier vide (commentaire seul) : il est réservé au pipeline TP7, pour écrire les événements dont le score de risque dépasse le seuil dans une collection `alerts` distincte du journal complet.

main.py — L'application Flask, chef d'orchestre [1 fichier]

main.py ne réinvente aucune logique de sécurité : il appelle les modules déjà décrits (auth/security, access/engine, db/connection) et se concentre sur l'expérience web — pages, session utilisateur, redirections, messages flash.

`_safe_find(collection_name, query, projection)`

Rôle

Tente une lecture MongoDB ; si elle échoue, relit directement le fichier CSV correspondant dans datasets/.

Pourquoi cette fonction existe séparément

Traduction directe de l'exigence du sujet : « journal persistant dans MongoDB ou CSV si MongoDB indisponible ». Centraliser ce repli ici évite de dupliquer un try/except MongoDB dans chaque route.

`_safe_find_one(collection_name, query)`

Rôle

Comme `_safe_find`, mais renvoie un seul document correspondant à la requête.

Pourquoi cette fonction existe séparément

Réutilise `_safe_find` plutôt que de dupliquer la logique de repli CSV — un seul comportement de secours à maintenir.

`_init_users() / _init_resources() / _init_logs()`

Rôle

Importent respectivement `users.csv`, `resources.csv` et `access_logs.csv` dans MongoDB, uniquement si la collection est encore vide.

Pourquoi cette fonction existe séparément

La vérification `count_documents({}) > 0` avant d'insérer évite de dupliquer les données à chaque redémarrage du serveur — sans cette garde, relancer l'application deux fois doublerait tous les utilisateurs.

`current_user()`

Rôle

Relit dans la base l'utilisateur correspondant à l'identifiant stocké en session Flask.

Pourquoi cette fonction existe séparément

Centralise "qui est connecté maintenant" : réutilisé par le `context_processor`, les décorateurs de garde, et plusieurs routes — un seul endroit si la façon de retrouver l'utilisateur change.

`_mfa_enabled(user) / _has_known_role(user) (privées)`

Rôle

Petites fonctions de lisibilité : est-ce que la MFA est activée pour cet utilisateur ? son rôle est-il défini dans la politique RBAC ?

Pourquoi cette fonction existe séparément

Sans elles, `login_page()` contiendrait des conditions plus longues et moins lisibles à l'intérieur d'un bloc déjà chargé de gérer formulaire, session et redirections.

`_get_totp_secret(user)`

Rôle

Renvoie le secret TOTP déjà stocké de l'utilisateur, ou celui généré temporairement en session si aucun n'existe encore.

Pourquoi cette fonction existe séparément

Gère le cas particulier d'un utilisateur qui n'a pas encore de secret persistant en base, sans dupliquer cette logique dans `_prepare_mfa_challenge` et `verify_otp_page`.

`_prepare_mfa_challenge(user)`

Rôle

Récupère ou génère le secret TOTP, calcule le code courant, tente l'envoi par email, sinon l'affiche dans le terminal.

Pourquoi cette fonction existe séparément

Regroupe toute la logique du "défi MFA" en un seul endroit, avec un repli explicite (affichage terminal) pour ne jamais bloquer un développeur qui teste sans configuration SMTP valide.

`_resource_access_preview(user, resource)`

Rôle

Calcule un statut informatif ("Lecture possible", "Refusé par rôle"...) pour l'affichage dans la liste des ressources, sans appeler `authorize()` ni écrire de log.

Pourquoi cette fonction existe séparément

Distinction importante : ceci est un **aperçu** d'affichage, pas une vraie tentative d'accès. La vraie vérification (avec journalisation) n'a lieu que sur la page de détail, en POST, via `authorize()`. Mélanger les deux aurait pollué le journal d'audit avec de faux "accès" qui ne sont en réalité que des survols de liste.

`inject_user()` – `@app.context_processor`

Rôle

Rend automatiquement `current_user` et `session_started` disponibles dans tous les templates Jinja, sans avoir à les repasser explicitement à chaque `render_template()`.

Pourquoi cette fonction existe séparément

Évite d'oublier de transmettre l'utilisateur courant à un template qui en aurait besoin (par exemple pour afficher le nom dans la barre de navigation) — un seul endroit à maintenir.

`role_required(*roles)` / `login_required`

Rôle

Deux décorateurs de garde : le premier exige d'être connecté ET d'avoir un rôle précis (ex. `admin_securite`), le second exige seulement d'être connecté, quel que soit le rôle.

Pourquoi cette fonction existe séparément

Séparer les deux permet de composer clairement chaque route : `/audit-logs` a besoin des deux (`@login_required` puis `@role_required("admin_securite")`), alors que `/dashboard` n'a besoin que d'être connecté — la déclaration des routes documente elle-même les permissions requises.

index() – GET /

Rôle

Redirige vers le tableau de bord si une session existe, sinon vers la page de connexion.

Pourquoi cette fonction existe séparément

Évite qu'un visiteur non connecté tombe sur une page blanche ou une erreur à la racine du site.

login_page() – GET/POST /login

Rôle

Vérifie le mot de passe (ou le mot de passe par défaut égal à l'identifiant, pour faciliter les tests), vérifie que le rôle est bien défini dans la politique, puis déclenche soit le défi MFA soit l'ouverture directe de session.

Pourquoi cette fonction existe séparément

C'est l'équivalent "site web" de auth/routes.py, mais avec des redirections et des messages flash au lieu de réponses JSON — logique dupliquée volontairement car les deux interfaces (API et site) ont des contrats de réponse différents.

_open_session(user, mfa_ok) (privée)

Rôle

Regroupe en un seul endroit tout ce qu'il faut écrire en session Flask après une authentification réussie : user_id, mfa_ok, heure de connexion, JWT, et nettoyage des valeurs temporaires.

Pourquoi cette fonction existe séparément

Sans cette fonction dédiée, il serait facile d'oublier un champ de session à un des deux endroits qui ouvrent une session (avec ou sans MFA) — bug classique de duplication de logique critique.

verify_otp_page() – GET/POST /verify-otp

Rôle

Deuxième étape MFA côté site web : vérifie le code TOTP saisi, puis ouvre la session complète.

Pourquoi cette fonction existe séparément

Symétrique de auth/routes.py:verify_otp(), mais orienté formulaire web plutôt que JSON.

logout() – GET /logout

Rôle

Vide entièrement la session Flask.

Pourquoi cette fonction existe séparément

Un simple session.clear() suffit puisque toute l'information de connexion (user_id, mfa_ok, token) est stockée uniquement en session — rien à révoquer côté serveur dans cette version du projet.

dashboard() – GET /dashboard

Rôle

Calcule, pour l'utilisateur connecté uniquement, le nombre total de tentatives, le nombre de refus, et la répartition par type de ressource consultée.

Pourquoi cette fonction existe séparément

Ces statistiques personnelles donnent à chaque utilisateur une visibilité sur sa propre activité — une forme de transparence qui incite aussi à repérer soi-même une anomalie sur son compte.

resources_page() – GET /resources

Rôle

Liste toutes les ressources, en annotant chacune du statut d'aperçu calculé par `_resource_access_preview()`.

Pourquoi cette fonction existe séparément

Permet à l'utilisateur de voir en un coup d'œil ce qu'il pourra probablement faire, avant de cliquer — meilleure expérience que de découvrir un refus après coup sur chaque ressource.

resource_detail(resource_id) – GET/POST /resources/<id>

Rôle

Affiche le détail d'une ressource ; en POST, exécute une vraie tentative d'accès via `authorize()` (donc journalisée), avec le résultat affiché en message flash.

Pourquoi cette fonction existe séparément

C'est ici, et seulement ici, qu'un accès "réel" à une ressource est tenté et journalisé — la séparation avec `resources_page()` (simple aperçu) est ce qui garde le journal d'audit propre et significatif.

audit_logs_page() – GET /audit-logs (admin_securite uniquement)

Rôle

Liste les 250 logs les plus récents et calcule le nombre de refus par utilisateur.

Pourquoi cette fonction existe séparément

Réservée à `admin_securite` via `role_required` — traduction directe de la politique RBAC qui réserve la lecture du `journal_acces` à ce rôle.

export_audit_csv() – GET /audit-logs/export.csv (admin_securite uniquement)

Rôle

Exporte l'intégralité du journal au format CSV téléchargeable.

Pourquoi cette fonction existe séparément

Complète `audit_logs_page()` (qui ne montre que les 250 plus récents) pour permettre une analyse complète hors ligne, par exemple dans le futur notebook de scoring de risque (TP7).

if __name__ == "__main__": ...

Rôle

Initialise les 3 collections MongoDB (avec repli silencieux si MongoDB est indisponible), puis démarre le serveur Flask.

Pourquoi cette fonction existe séparément

Le `try/except` autour des trois `_init_*`() garantit que l'application démarre quand même en mode "lecture CSV" si MongoDB n'est pas accessible au lancement — cohérent avec `_safe_find()`.

update_email.py — Script utilitaire ponctuel

Ce script n'est pas une route de l'application : c'est un petit script autonome, à exécuter manuellement une fois, pour mettre à jour l'adresse email d'un utilisateur de test directement en base (via `get_collection` déjà vu dans `db/connection.py`).

Pourquoi ne pas en faire une route Flask ? Parce que ce n'est pas un besoin récurrent de l'application — c'est une opération d'administration ponctuelle. Ajouter une route pour ce genre de besoin encombrerait inutilement l'application avec un endpoint qui ne sert qu'une fois.

Ce qu'il faut retenir de l'architecture globale

Un principe traverse tout le projet, visible dans presque chaque choix de conception ci-dessus : **chaque couche ne fait qu'une seule chose.**

auth/	Qui es-tu ?	Décider si tu as le droit d'agir
access/	As-tu le droit ?	Vérifier qui tu es (elle fait confiance au JWT)
audit/	Que s'est-il passé ?	Prendre une décision d'accès
db/	Comment parle-t-on à MongoDB ?	Contenir une règle métier
main.py	Quelle page afficher ?	Réimplémenter une règle de sécurité

Cette séparation est ce qui te permettra d'ajouter TP7 (streaming, scoring, alertes) sans toucher à `auth/` ni `access/` : le pipeline de supervision est une nouvelle couche qui **lit** le journal déjà produit par `audit/`, sans jamais avoir besoin de modifier la façon dont les décisions d'accès sont prises.